

Relazione Tecnica: Sistema Kanban Distribuito

Analisi critica su architettura ibrida e protocolli di coordinamento

Studente: Matteo Fortino — Anno Accademico: 2025/2026

1 Introduzione e Razionale Architettureale

Il progetto implementa una Kanban Board distribuita dove il coordinamento è affidato a un server centrale (*Lavagna*), mentre l'assegnazione dei task avviene tramite una negoziazione Peer-to-Peer (P2P) tra gli utenti. La scelta di questa architettura ibrida nasce dalla volontà di mantenere un'unica "sorgente della verità" per lo stato dei task, delegando però il carico decisionale della selezione dell'esecutore ai nodi periferici.

Sebbene funzionale, questa scelta presenta dei trade-off critici in termini di scalabilità e affidabilità che vengono analizzati nel dettaglio nei paragrafi successivi.

2 Analisi Critica: Il Problema della Scalabilità P2P

Il protocollo di negoziazione adottato è di tipo *all-to-all*. Quando un task entra in stato `TODO`, ogni peer riceve dal server la lista degli IP e delle porte di tutti gli altri partecipanti e avvia una fase di scambio costi.

- **Network Storm e Overhead TCP:** Il limite principale è la crescita quadratica $O(N^2)$ delle connessioni. In una rete con N utenti, ogni task genera una tempesta di tentativi di `connect()` simultanei. L'uso di TCP (protocollo connection-oriented) per scambi di dati così piccoli (un singolo intero) è inefficiente e introduce un overhead di handshake (SYN-ACK) che rallenta drasticamente la gara all'aumentare dei nodi.
- **Il Bias del Tie-Break:** Non potendo contare su un clock sincronizzato o su un arbitro, ho introdotto una regola di precedenza basata sul valore numerico della porta P2P. Sebbene risolva il problema della convergenza (tutti i peer arrivano alla stessa decisione), questo introduce un'ingiustizia algoritmica: i peer con identificativi più bassi avranno sempre la precedenza in caso di parità, un difetto che in un sistema reale porterebbe a uno sbilanciamento del carico di lavoro (*load imbalance*).

3 Gestione della Concorrenza e Multiplexing I/O

Il sistema utilizza la system call `select()` per gestire l'asincronia. Questa scelta è stata preferita ai thread per evitare problemi di *locking* e *race conditions* sulla memoria condivisa, ma non è priva di difetti.

- **Limiti della `select()`:** La `select()` ha un limite intrinseco nel numero di descrittori monitorabili (`FD_SETSIZE`) e costringe il kernel a scansionare l'intero set di descrittori ogni volta, con una complessità $O(N)$. In scenari ad alte prestazioni, sarebbe stato necessario evolvere verso `epoll()` (Linux) o `kqueue()` (BSD).
- **Consistenza Eventuale e Race Conditions:** Durante i test è emersa una vulnerabilità temporale. Se un peer invia il messaggio `MSG_CARD_DONE` nello stesso istante in cui il server fa scattare il timeout di espulsione, si rischia un'inconsistenza: il server potrebbe processare l'espulsione (rimettendo il task in `TODO`) e solo dopo ricevere il completamento. Ho risolto implementando una logica di "precedenza al lavoro": il server resetta forzatamente i timer di errore non appena riceve un pacchetto di completamento, privilegiando la *liveness* del sistema.

4 Robustezza e Fault Detection: Il Meccanismo Ping-Pong

Il rilevamento dei crash è affidato a un heartbeat (`MSG_PING`). Questa è forse la parte più critica del progetto dal punto di vista dell'affidabilità di rete.

- **Il problema dei Falsi Positivi:** Il timeout di 5 secondi per il `PONG` è una scelta arbitraria. In sistemi distribuiti reali, i ritardi di rete (jitter) possono superare facilmente questa soglia. Il rischio è che un utente perfettamente operativo venga espulso e il suo lavoro cancellato solo per un rallentamento momentaneo della rete.
- **Recupero dello Stato:** Il server implementa un meccanismo di *rollback*: quando un utente cade, le sue card `DOING` tornano in `TODO`. Tuttavia, non c'è memoria del lavoro già svolto; il sistema non supporta il "checkpointing", forzando un nuovo esecutore a ricominciare il task da zero.

5 Sicurezza e Integrità del Protocollo

Il progetto opera in un modello di "fiducia totale" (*Trusted Environment*), che rappresenta una grave vulnerabilità:

1. **Identity Spoofing:** Poiché il server identifica i peer solo tramite socket e porta, un utente malevolo potrebbe iniettare pacchetti fingendo di essere un altro peer per terminare task altrui.
2. **Man-in-the-Middle:** La mancanza di crittografia (TLS/SSL) rende i dati della board leggibili da chiunque sulla stessa sottorete, esponendo segreti industriali o descrizioni di task sensibili.

6 Conclusioni e Sviluppi Futuri

In conclusione, il progetto dimostra come un'architettura ibrida possa gestire un flusso di lavoro dinamico, ma evidenzia anche i limiti dei protocolli P2P non strutturati. Per rendere il sistema professionale, sarebbe necessario:

- Implementare un registro di log per la **persistenza** (attualmente, se la Lavagna cade, tutto lo stato va perduto).
- Utilizzare un algoritmo di elezione più scalabile (es. *Bully Algorithm* o *Ring-based*).
- Introdurre un sistema di autenticazione basato su token per validare i messaggi di completamento.

7 Istruzioni per la Compilazione

Il codice è organizzato per essere compilato tramite `Makefile`.

- Comando: `make` (genera gli eseguibili `lavagna` e `utente`).
- Test: Avviare il server e successivamente i client. Per simulare un guasto, terminare un client durante un task (`CTRL+C`) e attendere il ciclo di timeout della Lavagna per osservare il ripristino automatico della card.